An Introduction to Cost Estimation

Davide Falessi

Fundamental estimation questions

- How much **effort** is required to complete an activity?
- How much **calendar time** is needed to complete an activity?
- What is the **total cost** of an activity?

Software cost components

- Hardware and software costs
- Travel and training costs
- **Effort** costs (the dominant factor in most projects)
  - salaries of engineers involved in the project
  - social and insurance costs
- Effort costs must take overheads into account
  - costs of building, heating, lighting
  - costs of networking and communications
  - costs of shared facilities (e.g library, staff restaurant, etc.)

Cost vs. Price

- Estimates are made to discover the cost, to the developer, of producing a software system
- There is **not a simple relationship** between the development cost and the price charged to the customer
- Broader organisational, economic, political and business considerations influence the price charged

The role of programming language

	Analysis	Design	Coding	Testing	Documentation
Assembly code	3 weeks	5 weeks	8 weeks	10 weeks	2 weeks
High-level language	3 weeks	5 weeks	8 weeks	6 weeks	2 weeks

	Size	Effort	Productivity
Assembly code	5000 lines	28 weeks	714 lines/month
High-level language	1500 lines	20 weeks	300 lines/month

Estimation Techniques

- Algorithmic cost modelling
- Expert judgement
- Estimation by analogy
- Parkinson's Law
- Pricing to win

Algorithmic code modelling

- A **formulaic** approach based on historical cost information and which is generally based on the size of the software
- See COCOMO

Expert Judgement

- One or more experts in both software development and the application domain use their experience to predict software costs. Process iterates until some consensus is reached.
- **Advantages**: Relatively **cheap** estimation method. Can be accurate if experts have direct experience of similar systems
- **Disadvantages**: Very inaccurate if there are **no experts**!

Estimation by analogy

- The cost of a project is computed by comparing the project to a similar project in the same application domain
- **Advantages**: Accurate if project data **available**
- **Disadvantages**: Impossible if no comparable project has been tackled. Needs systematically maintained cost database

Parkinson's Law

- The project costs whatever resources are **available.** Time management is all psychological. We naturally pace ourselves to finish a project in the nick of time. The same task can take one hour or one week depending on how much time we give ourselves to complete it.
- **Advantages**: No overspend
- **Disadvantages**: System is usually unfinished

Pricing to win

- The project costs whatever the customer has to spend on it
- **Advantages**: You get the contract
- **Disadvantages**: The probability that the customer gets the system he or she wants is small. Costs do not accurately reflect the work required

Source

- SOFTWARE ENGINEERING, Ninth Edition, Ian Sommerville

An Introduction to Planning Poker

Davide Falessi

Agenda

- Intro
- Rules
- Details
- Tips

Introduction

- Planning Poker is an **agile estimation technique** which has become very popular in the last few years.
- Although the basics of the technique have been around for many years the refinements by Grenning made it **usable** by agile teams and they have taken full advantage.
- Planning Poker is extremely **simple** to play while also being **accurate** enough to use for agile planning.

Rules (1/2)

1. Each participant gets a deck of estimation cards representing a sequence of numbers. The most popular sequences involve doubling each card (0, ½, 1, 2, 4, 8, 16, etc.) or the Fibonacci sequence (1, 2, 3, 5, 8, 13, 21, etc.). The Fibonacci sequence is generally more popular.
2. The moderator presents one user story at a time to the team.
3. The Product Owner (or equivalent role) answers any questions the team might have about the story.
4. Each participant **privately selects a card** representing his or her estimate of the "size" for the user story.
5. **Size** represents a value taking into account **time, risk, complexity and any other relevant factors**.

Rules (2/2)

6. When everybody is ready with an estimate, all cards are presented **simultaneously**.
7. If there is consensus on a particular number then the size is recorded and the team moves to the next story.
8. In the (very likely) event that the estimates differ, the **highest and lowest estimators defend their estimates to the rest of the team**.
9. The group briefly debates the arguments.
10. A new round of estimation is made (steps 4 -9).
11. Continue **until consensus** has been reached and the moderator records the estimate.
12. Repeat for all stories.

Details

- There are very good reasons why most agile trainers and coaches promote its use.
  - Fosters **collaboration** by engaging entire team
  - Creates **consensus** estimate rather than having a single person driving the estimate
  - Exposes **issues early** through discussion of each user story

Tips

- **Those who actually could do the work are the ones that vote.** Too often agile teams have everyone vote even if they have no idea about the work involved in the story.
- **Managers don't vote.** Managers are usually incented to have the work take less time, so they often skew the vote too low.
- When there is a **tie** in the voting between two sizes which are consecutive, just **pick the larger** size and move on.

Tips

- **Use a timer of some sort to limit discussion.** This is self-explanatory. I usually like to limit discussions to no more than one minute.
- **Use an "I need a break" card.** Too often teams will be working hard playing planning poker and not realize some people on the team need a break. Having the "I need a break" card allows someone to draw everyone's attention to this.

Tips

- **Have the person creating the user stories meet with QA and Development leads prior to playing planning poker** to make sure the user stories have answers to the most obvious questions the team will ask. This allows the team to focus on the size, not spend time gathering information.
- **Remember the baseline.** Whatever the team picks as a baseline needs to be consistent from iteration to iteration. If an ideal day is a size 1 then all iterations need to use that reference point.

Source

- https://dzone.com/articles/introduction-planning-poker